

ExamForge — Aim Document

1. What is ExamForge?

ExamForge is a universal, secure, and tamper-resistant examination infrastructure designed to protect the complete examination lifecycle — from question creation to result publication.

It is not just an online exam portal or an AI proctoring tool. ExamForge is a zero-trust examination trust system that can support multiple types of exams.

- CBSE-style theoretical exams
- JEE-style computer-based MCQ exams
- NEET-style OMR-based exams
- UPSC-style descriptive written exams
- College semester exams
- Recruitment tests
- Scholarship exams
- Hybrid offline-online examinations

ExamForge is built around one central idea:

No single person, center, server, or authority should be able to compromise the examination process silently.

The system uses encrypted paper generation, controlled access, candidate verification, answer integrity checks, anonymous evaluation, and tamper-evident audit logs to make the examination process more trustworthy, transparent, and resistant to manipulation.

2. What does ExamForge solve?

Modern examination systems suffer from multiple trust and security problems. These problems are not limited to online exams; they also exist in traditional pen-paper examinations.

ExamForge aims to solve the following major problems:

2.1 Paper Leakage

Many examination failures happen before the exam even begins. Question papers may be leaked through insiders, storage breaches, printing centers, or transportation channels.

ExamForge reduces this risk by using:

- Encrypted question banks
- Blueprint-based paper generation
- Multiple equivalent paper sets
- Time-locked paper release
- Multi-party authorization
- Audit logs for every sensitive action

This ensures that no single person can easily access or leak the complete paper before the exam.

2.2 Exam Center Compromise

In offline exams, exam centers may become points of compromise due to early paper access, invigilator collusion, proxy candidates, or answer sheet tampering.

ExamForge addresses this through:

- Center-bound encrypted exam packages
- Time-window-based paper release
- Candidate identity verification
- Seat-level tracking
- Secure answer sheet identifiers
- Post-exam scan and hash verification

This makes exam center activity traceable and harder to manipulate.

2.3 Candidate Impersonation and Proxy Attempts

High-stakes exams often face proxy candidate issues, fake identity documents, or weak verification at exam centers.

ExamForge introduces a layered candidate verification system using:

- Admit card QR verification
- Government ID matching
- Live photo capture
- Signature verification
- Optional biometric support
- Entry and seat mapping logs

This creates a stronger identity chain from registration to answer submission.

2.4 OMR Sheet Tampering

In OMR-based exams, answer sheets can be replaced, modified, damaged, or disputed after the exam.

ExamForge solves this by assigning every OMR sheet a unique identity and storing a digital proof immediately after collection.

The system supports:

- Unique QR-coded OMR sheets
- Center-level scanning
- Image hash generation
- Bubble detection with confidence score
- Physical-digital sheet matching
- Tamper alerts if scanned copies mismatch

This creates proof that the submitted sheet was not silently changed later.

2.5 Written Answer Sheet Manipulation

In theoretical and descriptive exams, answer booklets may be mishandled, pages may be replaced, or evaluation may be biased.

ExamForge supports secure written-answer workflows through:

- QR-coded answer booklets
- Page-wise scanning
- Page hash verification
- Anonymous evaluator assignment
- Rubric-based marking
- Double evaluation for sensitive cases
- Evaluation audit trails

This improves fairness, traceability, and accountability in written examinations.

2.6 Result Manipulation

Even after exams are completed, results can be manipulated if answer data, marks, or evaluation records are editable without trace.

ExamForge prevents silent result manipulation through:

- Cryptographic answer receipts
- Tamper-evident audit ledger
- Evaluation locking

- Result verification before publication
- Reproducible result generation

Every major step is logged and chained, so unauthorized changes become detectable.

2.7 Lack of Trust and Transparency

Students, parents, institutions, and exam authorities often have limited visibility into whether the exam process was fair.

ExamForge provides verifiable trust by answering key questions:

- Was the paper generated fairly?
- Was the paper opened only at the correct time?
- Was the candidate identity verified?
- Was the answer submitted successfully?
- Was the answer sheet modified later?
- Was evaluation done anonymously and consistently?
- Was the final result generated from verified data?

The goal is not only to conduct exams, but to prove the integrity of the examination process.

3. For whom is ExamForge?

ExamForge is designed for organizations that conduct examinations where fairness, security, scale, and trust are important.

3.1 Schools and School Boards

ExamForge can support school-level and board-level exams such as CBSE-style theoretical examinations.

It helps with:

- Question paper security
 - Answer booklet tracking
 - Digital evaluation workflows
 - Reduced paper leakage risk
 - Transparent result generation
-

3.2 Competitive Examination Bodies

ExamForge is suitable for organizations conducting high-stakes exams similar to JEE, NEET, UPSC, SSC, banking, railway, defense, and recruitment examinations.

It helps with:

- Secure paper generation
 - Multiple paper sets
 - Candidate verification
 - OMR processing
 - Audit-backed result publishing
 - Reduced center-level malpractice
-

3.3 Universities and Colleges

Universities and colleges can use ExamForge for semester exams, entrance exams, lab exams, internal assessments, and hybrid evaluations.

It helps institutions modernize examination workflows without fully replacing their existing systems immediately.

3.4 Government Examination Authorities

Government bodies can use ExamForge as a secure digital infrastructure layer for large-scale examination management.

It can help improve:

- Public trust
 - Administrative traceability
 - Paper security
 - Evaluation transparency
 - Dispute handling
-

3.5 Private EdTech and Testing Platforms

Online test platforms, coaching institutes, scholarship platforms, and EdTech companies can integrate ExamForge to improve the security and credibility of their exams.

It can be used for:

- Online MCQ tests
 - Remote assessments
 - Scholarship exams
 - Mock test integrity
 - AI-assisted evaluation workflows
-



4. In what form is ExamForge delivered?

ExamForge can be delivered in multiple forms depending on the scale and requirements of the examination authority.

The proposed delivery model is modular so that institutions can adopt only the parts they need.

4.1 ExamForge as a Complete SaaS Platform

In this model, ExamForge is delivered as a cloud-based web platform.

The examination authority can log in and manage the full exam lifecycle from one dashboard.

The SaaS platform includes:

- Admin dashboard
- Question bank management
- Paper generation system
- Candidate management
- Exam scheduling
- Secure paper release
- OMR upload and scanning
- Written answer evaluation
- Result generation
- Audit dashboard

This model is best for schools, colleges, coaching institutions, and smaller examination bodies that need a ready-to-use solution.

4.2 ExamForge as Backend Infrastructure

In this model, ExamForge works as a secure backend system that other platforms can integrate with.

It exposes APIs for:

- Exam creation
- Question bank access
- Paper generation
- Paper encryption
- Candidate verification
- Answer submission
- OMR processing
- Evaluation locking
- Result verification
- Audit log validation

This model is best for EdTech companies, universities, and testing platforms that already have their own frontend or student portal.

4.3 ExamForge as Middleware

ExamForge can also work as a middleware layer between existing examination software and institutional systems.

In this form, it does not replace the entire exam platform. Instead, it adds trust, security, and verification features to existing workflows.

Middleware capabilities include:

- Secure paper packaging
- Time-locked paper release
- Audit logging
- Answer hash generation
- Result integrity verification
- OMR/written answer sheet tracking
- Anonymous evaluation layer

This is best for large institutions that already have examination software but need stronger security and auditability.

4.4 ExamForge as On-Premise Deployment

For highly sensitive exams, ExamForge can be deployed inside the institution's own infrastructure instead of the public cloud.

This model is suitable for:

- Government exams
- Defense-related exams
- High-stakes national exams

- Universities with strict data policies
- Organizations requiring full data control

On-premise deployment provides:

- Local database control
 - Private network operation
 - Institution-owned encryption keys
 - Offline exam center support
 - Internal audit control
-

4.5 ExamForge as Hybrid Infrastructure

For national or large-scale examinations, ExamForge can be deployed as a hybrid system.

In this model:

- Central authorities manage exam creation and paper generation.
- Exam centers receive encrypted packages.
- Candidate verification can happen locally.
- Answer data is uploaded securely.
- Evaluation can happen centrally.
- Audit verification can happen across both local and central systems.

This model is best for large public examinations where offline centers, digital systems, and centralized result processing must work together.

Final Aim Statement

The aim of ExamForge is to design and build a universal examination trust infrastructure that makes examinations more secure, fair, transparent, and tamper-resistant across multiple formats.

ExamForge protects the entire examination pipeline — question creation, paper generation, exam delivery, candidate verification, answer capture, evaluation, and result publication — using a zero-trust architecture, cryptographic verification, audit trails, and modular deployment options.

The long-term vision of ExamForge is to become a secure examination layer that can be used by schools, universities, competitive exam bodies, government agencies, and EdTech platforms to conduct examinations whose integrity can be verified, not merely trusted.

ExamForge — Solution Document

1. Project Overview

ExamForge is a universal zero-trust examination infrastructure designed to secure the complete examination lifecycle across MCQ, OMR, and written examination formats.

The system provides a secure digital layer for exam creation, paper generation, controlled paper release, candidate verification, answer capture, evaluation, result generation, and audit verification.

ExamForge is built for examinations where trust, fairness, and tamper-resistance are critical. It can support school board exams, competitive entrance exams, recruitment tests, university exams, and hybrid offline-online assessments.

The core objective of ExamForge is simple:

To make every critical step of an examination verifiable, traceable, and resistant to silent compromise.

2. Problem Statement

Existing examination systems face several security and trust issues:

- Question papers may be leaked before the exam.
- Exam centers may access papers early or manipulate exam processes.
- Proxy candidates may appear in place of real candidates.
- OMR sheets and written answer booklets may be replaced or modified.
- Digital answer records may be changed after submission.
- Evaluation may be biased, inconsistent, or manipulated.
- Final results may be altered without a transparent audit trail.
- Students and institutions often have no independent way to verify exam integrity.

Most existing systems solve only one part of the problem, such as online proctoring or digital testing. ExamForge solves the larger problem by securing the entire examination pipeline.



3. Proposed Solution

ExamForge proposes a modular, zero-trust examination platform that combines:

- Secure question bank management
- Blueprint-based paper generation
- Encrypted exam packaging
- Time-locked paper release
- Candidate identity verification
- MCQ, OMR, and written answer capture
- Tamper-evident audit logs
- Anonymous and rubric-based evaluation
- Result integrity verification
- Modular SaaS, backend, middleware, and on-premise deployment models

The system assumes that no single person, center, server, or authority should be fully trusted. Instead, every sensitive action is recorded, verified, and cryptographically protected.



4. Solution Philosophy

ExamForge follows four major principles.

4.1 Zero-Trust Examination Design

Every actor in the system is treated as potentially untrusted unless verified.

This includes:

- Admins
- Question creators
- Reviewers
- Exam centers
- Invigilators
- Evaluators
- Candidates
- Servers
- Databases

No single actor should be able to compromise the full exam process alone.

4.2 Tamper-Evident, Not Just Tamper-Resistant

ExamForge does not only try to prevent tampering. It also ensures that if tampering occurs, it becomes detectable.

This is done using:

- Hashing
 - Digital signatures
 - Audit chains
 - Submission receipts
 - Scan hashes
 - Result verification checks
-

4.3 Format-Agnostic Exam Support

ExamForge is designed to support different kinds of examinations instead of being limited to only online tests.

Supported formats include:

- Computer-based MCQ exams
 - OMR-based paper exams
 - Descriptive written exams
 - Hybrid exams
 - Institution-level tests
 - Competitive exams
-

4.4 Modular Adoption

Institutions do not need to replace their full examination system immediately.

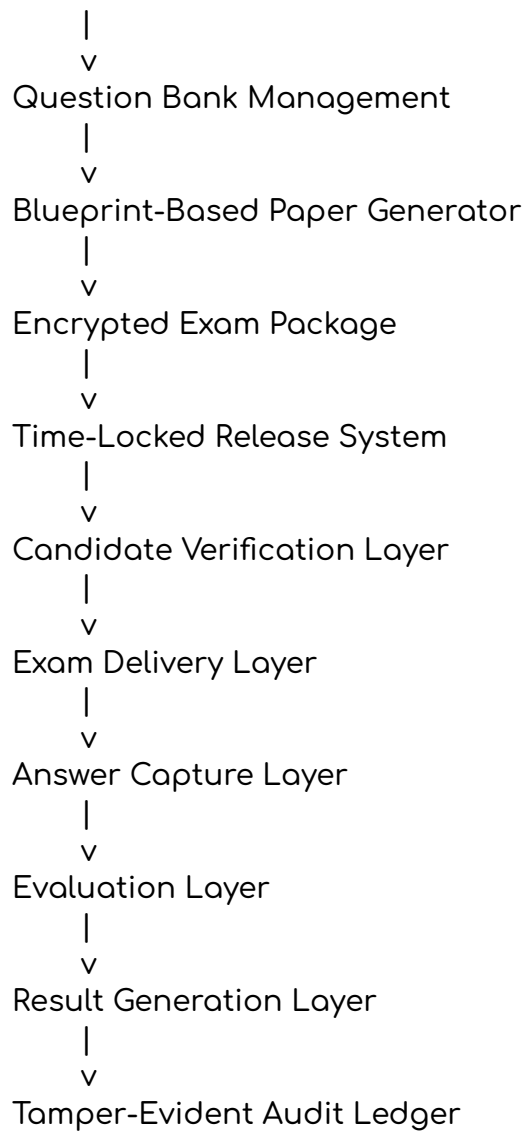
ExamForge can be adopted as:

- A complete SaaS platform
 - A backend infrastructure layer
 - A middleware security layer
 - An on-premise deployment
 - A hybrid central-local examination system
-

5. System Architecture

5. System Architecture

Admin / Exam Authority



5.2 Main Components

ExamForge is divided into the following core modules:

1. Question Bank Module
2. Paper Blueprint Module
3. Paper Generation Engine
4. Encryption and Release Module
5. Candidate Verification Module
6. Exam Delivery Module
7. Answer Capture Module
8. OMR Processing Module
9. Written Answer Evaluation Module
10. Result Generation Module
11. Audit Ledger Module
12. Admin and Monitoring Dashboard

6. Core Modules

6.1 Question Bank Module

The Question Bank Module stores and manages all questions securely.

Each question contains:

- Question ID
- Subject
- Topic
- Difficulty level
- Question type
- Marks
- Expected solving time
- Language
- Author ID
- Reviewer ID
- Approval status
- Usage history
- Version number
- Question hash

Supported question types:

- Single-correct MCQ
- Multi-correct MCQ
- Numerical answer
- Short answer
- Long answer
- Essay
- Case study
- Diagram-based question
- OMR question
- Practical question

Security Features

- Questions are stored in encrypted form.
- Every question update creates a new version.
- Only approved questions can enter the paper generation pool.
- Question access is logged.
- Question usage history prevents repeated or predictable papers.

6.2 Paper Blueprint Module

The Paper Blueprint Module defines the structure of an examination paper.

A blueprint may specify:

- Total marks
- Total questions
- Subject distribution
- Topic distribution
- Difficulty ratio
- Question type ratio
- Section-wise rules
- Negative marking rules
- Time limit
- Language options

Example blueprint:

Total Questions: 100
Total Marks: 400

Physics: 33 questions
Chemistry: 34 questions
Biology: 33 questions

Difficulty:
Easy: 25%
Medium: 50%
Hard: 25%

Question Type:
MCQ: 80%
Numerical: 20%

The blueprint ensures that generated papers are balanced, fair, and consistent.

6.3 Paper Generation Engine

The Paper Generation Engine creates exam papers from the approved question bank using the defined blueprint.

Process

1. Load approved questions.
2. Filter by subject, topic, difficulty, and type.
3. Select questions according to blueprint constraints.
4. Generate one or more equivalent paper sets.
5. Shuffle question order.
6. Shuffle MCQ option order.
7. Calculate paper difficulty score.
8. Validate paper against blueprint.
9. Lock generated paper.
10. Create paper hash and audit event.

Output

The output is a locked paper object containing:

- Paper ID
 - Exam ID
 - Set ID
 - Question list
 - Option order
 - Answer key reference
 - Blueprint compliance score
 - Difficulty score
 - Paper hash
-

6.4 Encryption and Time-Locked Release Module

After generation, the paper is encrypted and cannot be opened until the authorized release time.

Features

- AES-based paper encryption
- Paper hash generation
- Time-window validation
- Center-specific package binding
- Role-based release permission
- Multi-party approval support
- Emergency access protocol

Purpose

This module prevents early paper access and reduces the risk of pre-exam paper leaks.

For offline centers, ExamForge can generate encrypted exam packages that are bound to:

- Exam ID
- Center ID
- Date and time
- Authorized officer IDs
- Device ID
- Package checksum

Before release, the system verifies that the correct center, correct time, and correct authority are present.

6.5 Candidate Verification Module

This module verifies candidate identity before allowing exam participation.

Supported Verification Methods

- Admit card QR code
- Candidate registration ID
- Government ID check
- Live photo capture
- Face match with stored photo
- Signature verification
- Optional biometric verification
- Seat mapping

Candidate Verification Record

Each candidate verification creates a record containing:

- Candidate anonymous exam ID
- Verification timestamp
- Center ID
- Seat ID
- Invigilator ID
- Verification method used
- Verification status
- Verification hash

This creates a traceable identity chain for the examination.

6.6 Exam Delivery Module

The Exam Delivery Module delivers the paper to candidates depending on the exam format.

For Computer-Based Exams

The system provides:

- Candidate login
- Exam timer
- Question navigation
- Answer saving
- Mark for review
- Auto-save
- Final submission
- Submission receipt

For OMR-Based Exams

The system provides:

- Printable OMR sheets
- Unique QR code per sheet
- Paper set mapping
- Candidate-sheet binding
- Post-exam scan upload

For Written Exams

The system provides:

- QR-coded answer booklets
 - Page-wise identifiers
 - Candidate-booklet mapping
 - Post-exam scanning
 - Digital evaluation interface
-

6.7 Answer Capture Module

This module captures and locks candidate responses.

For CBT / MCQ Exams

Every candidate action is captured as an event:

- Question opened
- Answer selected
- Answer changed
- Question marked for review
- Answer saved
- Exam submitted

Each event is linked to the previous event hash, creating a tamper-evident answer timeline.

For OMR Exams

The OMR sheet is scanned after the exam. The system stores:

- OMR image
- Image hash
- Detected answers
- Bubble confidence score
- Manual review status
- Final locked response

For Written Exams

The answer booklet is scanned page-wise. The system stores:

- Booklet ID
- Page ID
- Page image
- Page hash
- Candidate anonymous ID
- Evaluation status

6.8 OMR Processing Module

The OMR Processing Module reads scanned OMR sheets and extracts answers.

Process

1. Upload scanned OMR image.
2. Detect QR code and sheet ID.
3. Align image using reference markers.
4. Detect bubble regions.
5. Calculate fill percentage.
6. Determine selected option.
7. Generate confidence score.
8. Flag ambiguous answers.

9. Store response hash.
10. Lock final OMR response.

Example Output

Candidate: ANON-9241

Sheet ID: OMR-7712

Set ID: B

Q1: A | Confidence: 98%

Q2: D | Confidence: 96%

Q3: Ambiguous | Confidence: 52% | Manual Review Required

Q4: C | Confidence: 94%

This prevents silent OMR replacement or answer modification.

6.9 Written Answer Evaluation Module

The Written Answer Evaluation Module supports secure descriptive answer checking.

Features

- Anonymous answer copies
- Evaluator assignment
- Rubric-based marking
- Question-wise marks entry
- Optional double evaluation
- AI-assisted feedback support
- Evaluation consistency checks
- Marks locking
- Evaluation audit logs

Evaluation Philosophy

ExamForge does not replace human evaluators. Instead, it provides a secure and structured system where evaluators can mark answers fairly and transparently.

AI may assist in:

- Rubric matching
- Missing point detection
- Similarity checking
- Plagiarism detection
- Evaluation consistency analysis

The final decision remains with human evaluators.

6.10 Result Generation Module

The Result Generation Module calculates final marks only after verifying that all required integrity checks have passed.

Verification Before Result Generation

The system verifies:

- Paper hash
- Answer submission hash
- OMR image hash
- Written page hashes
- Evaluation lock status
- Audit chain validity
- Answer key version
- Candidate identity record
- Result calculation rules

Only after successful verification does the system generate the final result.

Result Output

Each result contains:

- Candidate anonymous ID
- Exam ID
- Section-wise marks
- Total marks
- Rank or qualification status
- Result hash
- Result generation timestamp
- Verification status

6.11 Tamper-Evident Audit Ledger

Every sensitive action in ExamForge is stored in an audit ledger.

Logged Events

- Question created

- Question reviewed
- Question approved
- Paper generated
- Paper encrypted
- Paper released
- Candidate verified
- Candidate started exam
- Candidate submitted answer
- OMR scanned
- Answer booklet uploaded
- Evaluator assigned
- Marks entered
- Marks locked
- Result generated
- Result published

Audit Log Structure

Each audit event contains:

- Event ID
- Actor ID
- Action type
- Timestamp
- Payload hash
- Previous event hash
- Current event hash
- Digital signature

This creates a hash chain. If any past event is modified, the chain becomes invalid.

7. Role-Based Access Control

ExamForge uses role-based access control to limit what each user can do.

Roles

Super Admin

- Manage institutions
- Configure system policies
- View high-level audit records

Exam Controller

- Create exams
- Approve blueprints
- Generate papers
- Authorize paper release
- Publish results

Question Author

- Create questions
- Edit own questions
- Submit questions for review

Reviewer

- Review questions
- Approve or reject questions
- Suggest corrections

Center Officer

- Verify candidates
- Release center-specific paper packages
- Upload OMR/written scans

Invigilator

- Mark attendance
- Verify seat mapping
- Report suspicious activity

Evaluator

- Evaluate anonymized written answers
- Submit question-wise marks
- Lock evaluation

Candidate

- Appear for exam
- Submit answers
- Download submission receipt
- View result when published

Auditor

- View audit records

- Verify result integrity
 - Investigate disputes
-

8. Security Design

8.1 Encryption

ExamForge encrypts sensitive data such as:

- Question content
- Generated papers
- Answer keys
- Candidate documents
- Exam packages

Recommended encryption:

- AES-256-GCM for encrypted data storage
 - TLS for data transmission
 - Secure key management for encryption keys
-

8.2 Hashing

Hashing is used to verify integrity.

The system hashes:

- Questions
- Papers
- Candidate responses
- OMR images
- Written answer pages
- Evaluation records
- Result files
- Audit logs

Recommended hashing:

- SHA-256 or SHA-3
-

8.3 Digital Signatures

Digital signatures are used to prove authenticity of critical records.

Signed records include:

- Generated paper
- Paper release approval
- Candidate submission receipt
- OMR scan record
- Marks lock event
- Published result

Recommended signature algorithms:

- Ed25519
 - ECDSA
-

8.4 Time-Locked Release

Papers cannot be decrypted before the scheduled exam time.

The release system checks:

- Exam schedule
 - Center authorization
 - User role
 - Required approvals
 - Package integrity
 - Device/session validity
-

8.5 Audit Chain

Audit logs are hash-chained.

Example:

Event 1 Hash = Hash(Event 1 Data)

Event 2 Hash = Hash(Event 2 Data + Event 1 Hash)

Event 3 Hash = Hash(Event 3 Data + Event 2 Hash)

If Event 1 is modified later, Event 2 and Event 3 no longer match. This makes tampering detectable.



9. Workflow

9.1 Admin Workflow

1. Create exam.
 2. Add subjects and syllabus.
 3. Define paper blueprint.
 4. Add or import questions.
 5. Review and approve questions.
 6. Generate paper sets.
 7. Encrypt generated paper.
 8. Schedule paper release.
 9. Monitor exam progress.
 10. Verify submissions.
 11. Publish result after audit validation.
-

9.2 Candidate Workflow

1. Register for exam.
 2. Receive admit card.
 3. Verify identity.
 4. Start exam.
 5. Submit answers.
 6. Receive submission receipt hash.
 7. View result after publication.
-

9.3 OMR Workflow

1. Generate QR-coded OMR sheets.
 2. Bind OMR sheet to candidate.
 3. Conduct exam offline.
 4. Scan OMR after exam.
 5. Extract answers using OMR detection.
 6. Flag ambiguous bubbles.
 7. Lock final response.
 8. Generate marks after answer key verification.
-

9.4 Written Exam Workflow

1. Generate QR-coded answer booklets.
 2. Bind booklet to candidate.
 3. Conduct written exam.
 4. Scan answer booklet page-wise.
 5. Store page hashes.
 6. Assign anonymized copies to evaluators.
 7. Mark using rubric.
 8. Lock marks.
 9. Generate result after audit validation.
-



10. Data Model

10.1 Main Entities

User
Role
Institution
Exam
Subject
Question
QuestionVersion
PaperBlueprint
GeneratedPaper
EncryptedPackage
Candidate
CandidateVerification
ExamSession
CandidateAnswer
OMRSheet
OMRScan
WrittenBooklet
WrittenPage
Evaluation
Result
AuditLog

10.2 Example Tables

exams

id

title
exam_type
institution_id
start_time
end_time
status
created_by
created_at

questions

id
subject
topic
difficulty
question_type
marks
encrypted_content
status
author_id
reviewer_id
version
content_hash
created_at

generated_papers

id
exam_id
set_id
blueprint_id
encrypted_paper
paper_hash
difficulty_score
status
created_at

candidate_answers

id
candidate_id
exam_id
question_id
selected_answer
answer_hash
previous_event_hash
created_at

audit_logs

id
actor_id
action
payload_hash
previous_hash
current_hash
signature
timestamp

11. API Design

11.1 Exam APIs

POST /api/exams
GET /api/exams/{exam_id}
POST /api/exams/{exam_id}/blueprint
POST /api/exams/{exam_id}/generate-paper
POST /api/exams/{exam_id}/encrypt-paper
POST /api/exams/{exam_id}/release-paper

11.2 Question Bank APIs

POST /api/questions
GET /api/questions
GET /api/questions/{question_id}
POST /api/questions/{question_id}/review
POST /api/questions/{question_id}/approve

11.3 Candidate APIs

POST /api/candidates/register
POST /api/candidates/verify
POST /api/exam-session/start
POST /api/exam-session/save-answer
POST /api/exam-session/submit
GET /api/candidates/{candidate_id}/result

11.4 OMR APIs

POST /api/omr/generate-sheet
POST /api/omr/upload-scan
POST /api/omr/process
POST /api/omr/manual-review
POST /api/omr/lock-response

11.5 Written Exam APIs

POST /api/written/booklet/create
POST /api/written/booklet/upload-page
POST /api/written/evaluation/assign
POST /api/written/evaluation/submit-marks
POST /api/written/evaluation/lock

11.6 Audit APIs

GET /api/audit/logs
GET /api/audit/verify-chain
GET /api/audit/result/{result_id}/verify



12. Technology Stack

12.1 Frontend

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- React Hook Form
- Chart.js or Recharts

12.2 Backend

- FastAPI
- Python
- SQLAlchemy
- PostgreSQL

- Redis for caching and session locks

12.3 AI / Computer Vision

- OpenCV for OMR processing
- Optional OCR for written page indexing
- Optional LLM/RAG for rubric assistance
- Optional plagiarism/similarity detection

12.4 Security

- AES-256-GCM encryption
- SHA-256 hashing
- Ed25519 digital signatures
- JWT authentication
- Role-based access control
- TLS communication

12.5 Deployment

- Docker
 - PostgreSQL
 - Nginx
 - Cloud deployment for SaaS
 - On-premise deployment for sensitive institutions
-



13. Hackathon MVP Scope

The hackathon MVP should demonstrate the core trust pipeline without overbuilding the entire national-scale system.

Must-Have MVP Features

1. Admin dashboard
2. Question bank
3. Blueprint-based paper generation
4. Paper encryption and release simulation
5. Candidate MCQ exam portal
6. Candidate answer submission hash
7. OMR upload and basic bubble detection
8. Written answer upload with anonymous evaluator view
9. Tamper-evident audit ledger

10. Result generation with audit verification

Nice-to-Have Features

1. Multiple paper sets
 2. QR-coded OMR sheet generation
 3. QR-coded written answer booklet
 4. Manual review for ambiguous OMR answers
 5. Result verification page
 6. AI-assisted written answer rubric matching
 7. Admin analytics dashboard
-

Avoid in MVP

1. Real biometric hardware
 2. Full blockchain
 3. Full AI essay grading
 4. National-scale infrastructure
 5. Complex proctoring system
 6. Real government ID integration
-

14. Demo Flow

Step 1: Create Exam

Admin creates a sample exam:

Exam Name: National Scholarship Test

Mode: Hybrid

Sections: MCQ + OMR + Written

Step 2: Add Question Bank

Admin adds approved questions with metadata:

Subject: Science

Topic: Electricity

Difficulty: Medium
Type: MCQ
Marks: 4

Step 3: Generate Paper

System generates paper based on blueprint.

Output:

Paper Generated
Set A Created
Set B Created
Difficulty Balanced
Paper Hash Generated
Audit Event Created

Step 4: Encrypt Paper

System encrypts the paper and shows:

Paper Status: Encrypted
Access Before Time: Denied

Step 5: Release Paper

At the allowed release time, the paper becomes available.

Output:

Release Verified
Paper Decrypted
Audit Chain Updated

Step 6: Candidate Takes MCQ Exam

Candidate answers questions and submits.

Output:

Submission Successful
Submission Receipt Hash: EXF-91A7F3C2

Step 7: OMR Processing

Admin uploads a sample OMR sheet.

System detects answers:

Q1: A | Confidence: 97%
Q2: C | Confidence: 94%
Q3: Ambiguous | Manual Review Required

Step 8: Written Evaluation

Admin uploads a written answer page.

Evaluator sees anonymized copy and enters marks using rubric.

Output:

Candidate Identity Hidden
Marks Locked
Evaluation Hash Generated

Step 9: Result Verification

Before publishing result, system verifies:

Paper Hash: Valid
Submission Hash: Valid
OMR Scan Hash: Valid
Evaluation Lock: Valid
Audit Chain: Valid

Then result is published.



15. Expected Impact

ExamForge can improve examination systems by:

- Reducing paper leakage risk
 - Preventing silent answer tampering
 - Improving candidate identity verification
 - Making OMR and written sheets traceable
 - Increasing fairness in evaluation
 - Creating transparent audit trails
 - Improving trust among students and institutions
 - Supporting both online and offline exam formats
-



16. Future Scope

Future versions of ExamForge can include:

- Threshold cryptography for multi-party key release
 - Hardware security module support
 - Offline-first center application
 - Advanced face verification
 - Secure printing integration
 - AI-based malpractice anomaly detection
 - Large-scale result analytics
 - Blockchain anchoring for public audit checkpoints
 - Multi-language question paper generation
 - National-level exam center monitoring
 - Secure dispute resolution portal
-



17. Conclusion

ExamForge is a universal examination security infrastructure designed to protect the full lifecycle of exams.

Instead of focusing only on online proctoring or question generation, ExamForge secures the complete chain:

Question → Paper → Candidate → Answer → Evaluation → Result → Audit

By combining zero-trust design, encryption, identity verification, answer hashing, anonymized evaluation, and tamper-evident audit logs, ExamForge makes examination compromise significantly harder and easier to detect.

The final goal is to create an examination system where fairness is not merely assumed, but technically verifiable.

ExamForge — Design Document

1. Document Purpose

This document defines the technical and product design of ExamForge, a universal zero-trust examination infrastructure for MCQ, OMR, and written examinations.

The purpose of this document is to describe how ExamForge is structured, how its major systems communicate, how data flows through the platform, how core modules are designed, and how users interact with the system through the UI.

This document covers:

- High-Level Design
 - Low-Level Design
 - UI/UX Design
 - Database Design
 - API Design
 - Security Design
 - MVP Scope
-

2. Product Summary

2.1 What is ExamForge?

ExamForge is a secure examination infrastructure that protects the complete examination lifecycle:

Question Creation → Paper Generation → Paper Encryption → Paper Release → Candidate Verification → Exam Attempt → Answer Capture → Evaluation → Result Generation → Audit Verification

It supports:

- Computer-based MCQ exams
- OMR-based exams
- Descriptive written exams
- Hybrid offline-online exams

The system is built using a zero-trust model where every major action is verified, logged, and made tamper-evident.

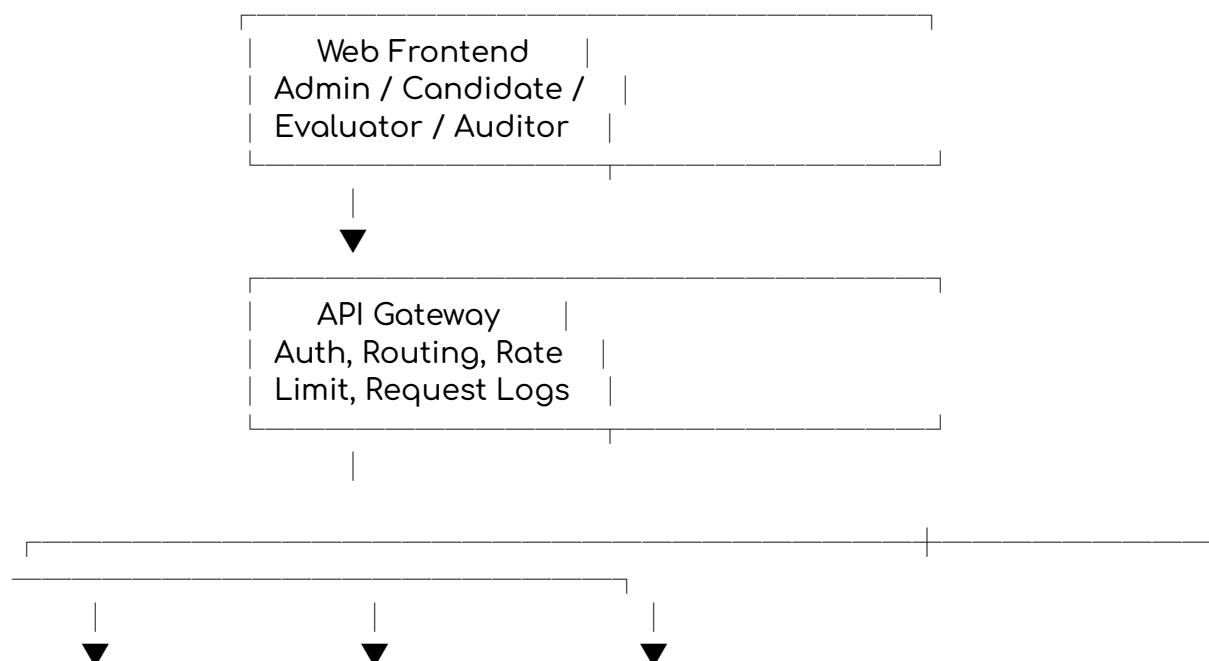
2.2 Core Product Goals

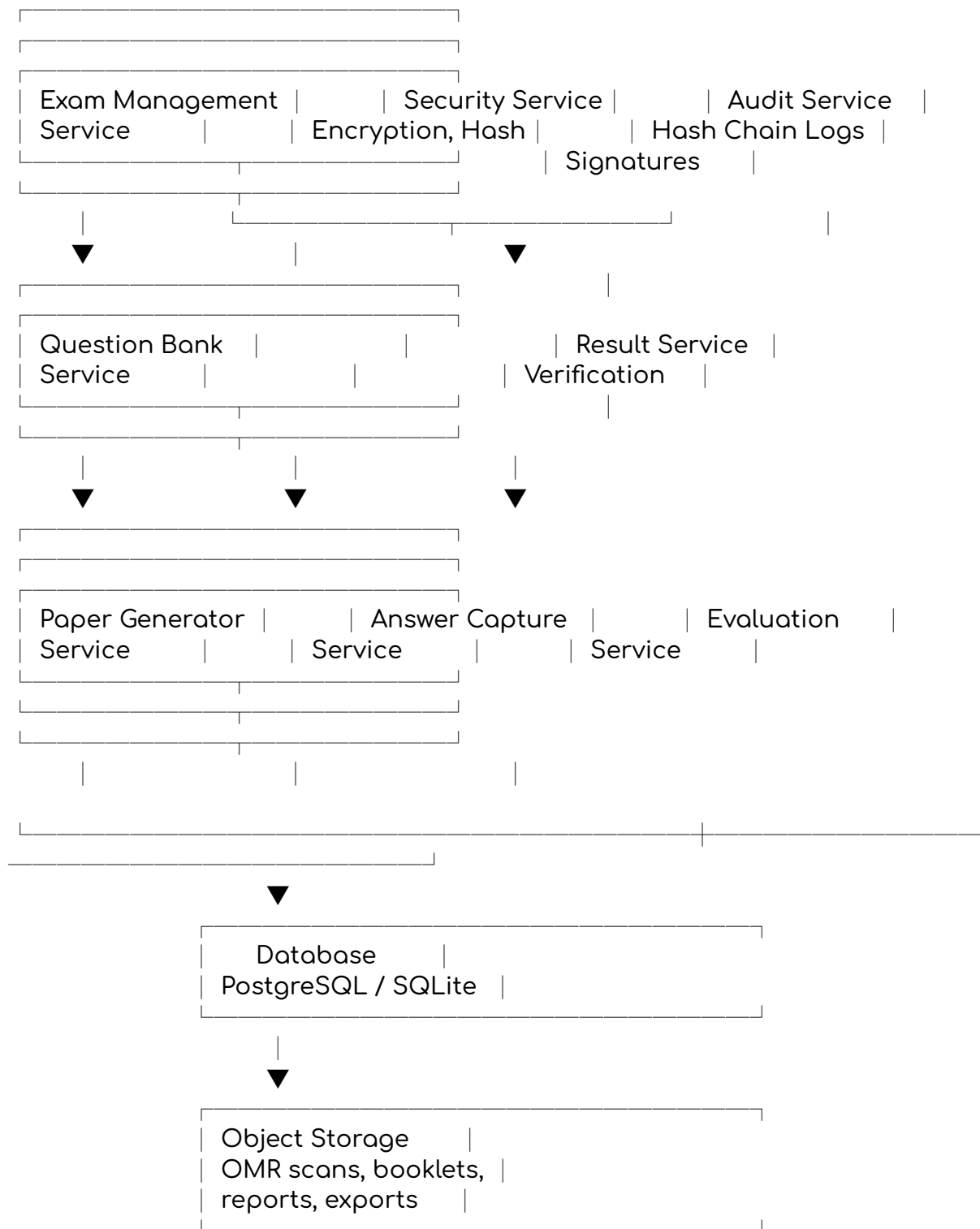
ExamForge aims to:

1. Prevent early paper leakage.
 2. Reduce exam center compromise.
 3. Verify candidate identity.
 4. Protect OMR and written answer sheets from tampering.
 5. Make evaluation traceable and fair.
 6. Generate results only from verified records.
 7. Provide a complete audit trail for dispute resolution.
 8. Support multiple exam formats through one modular platform.
-

3. High-Level Design

3.1 High-Level System Architecture





3.2 Major Subsystems

ExamForge is divided into the following major subsystems:

1. Authentication and Role Management

2. Exam Management System
 3. Question Bank System
 4. Paper Blueprint System
 5. Paper Generation Engine
 6. Encryption and Release System
 7. Candidate Verification System
 8. Exam Delivery System
 9. Answer Capture System
 10. OMR Processing System
 11. Written Evaluation System
 12. Result Generation System
 13. Tamper-Evident Audit Ledger
 14. Admin Monitoring Dashboard
-

3.3 User Roles

Role	Description
Super Admin	Manages institutions, policies, and platform configuration
Exam Controller	Creates exams, approves blueprints, generates papers, publishes results
Question Author	Creates and submits questions
Reviewer	Reviews and approves questions
Center Officer	Manages exam center operations
Invigilator	Verifies candidates and exam conduct
Candidate	Appears for exams
Evaluator	Checks written answers anonymously
Auditor	Verifies logs, results, and disputes

3.4 High-Level Data Flow

1. Question author creates questions.
2. Reviewer approves questions.
3. Exam controller creates exam and blueprint.
4. Paper generator selects questions from approved bank.
5. Generated paper is hashed, locked, and encrypted.
6. Paper is released only during authorized time window.

7. Candidate identity is verified.
 8. Candidate attempts exam or submits physical OMR/written sheet.
 9. Answers are captured and hashed.
 10. Evaluators mark written answers anonymously.
 11. Result service verifies all integrity checks.
 12. Result is generated and published.
 13. Auditor can verify full audit chain.
-

4. Low-Level Design

4.1 Authentication and Authorization Module

Responsibility

This module handles login, authentication, user sessions, and role-based permissions.

Main Functions

```
register_user()
login_user()
verify_token()
assign_role()
check_permission()
logout_user()
```

Access Rules

Example:

Exam Controller:

- Can create exam
- Can generate paper
- Can release paper
- Can publish result

Question Author:

- Can create question
- Can edit own question
- Cannot approve question
- Cannot generate paper

Evaluator:

- Can view assigned anonymized answers

- Can submit marks
- Cannot see candidate identity

Token Design

Use JWT with:

user_id
role
institution_id
session_id
expiry
signature

4.2 Question Bank Module

Responsibility

Stores questions with metadata, approval status, version control, and hashes.

Question Object

```
{  
  "id": "QST-1001",  
  "subject": "Physics",  
  "topic": "Electricity",  
  "difficulty": "Medium",  
  "question_type": "MCQ_SINGLE",  
  "marks": 4,  
  "expected_time_seconds": 90,  
  "language": "English",  
  "encrypted_content": "...",  
  "correct_answer_encrypted": "...",  
  "status": "APPROVED",  
  "author_id": "USR-21",  
  "reviewer_id": "USR-42",  
  "version": 3,  
  "content_hash": "sha256_hash",  
  "created_at": "timestamp",  
  "updated_at": "timestamp"  
}
```

Question Status

DRAFT

SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
ARCHIVED

Core Functions

create_question()
update_question()
submit_question_for_review()
review_question()
approve_question()
reject_question()
archive_question()
get_approved_questions()
calculate_question_hash()

4.3 Paper Blueprint Module

Responsibility

Defines rules for paper generation.

Blueprint Object

```
{  
  "id": "BLP-001",  
  "exam_id": "EXM-001",  
  "total_marks": 400,  
  "total_questions": 100,  
  "duration_minutes": 180,  
  "subject_distribution": {  
    "Physics": 33,  
    "Chemistry": 34,  
    "Biology": 33  
  },  
  "difficulty_distribution": {  
    "Easy": 25,  
    "Medium": 50,  
    "Hard": 25  
  },  
  "question_type_distribution": {  
    "MCQ_SINGLE": 80,  
    "NUMERICAL": 20  
  },  
}
```

```
"negative_marking": true,  
"negative_marks": 1  
}
```

Core Functions

```
create_blueprint()  
validate_blueprint()  
update_blueprint()  
lock_blueprint()  
get_blueprint_by_exam()
```

4.4 Paper Generation Engine

Responsibility

Generates balanced papers from the approved question bank according to the blueprint.

Algorithm

Input:

- exam_id
- blueprint_id
- approved question pool
- number_of_sets

Steps:

1. Load blueprint.
2. Load approved questions.
3. Group questions by subject, topic, difficulty, and type.
4. Sample questions according to blueprint constraints.
5. Validate total question count.
6. Validate total marks.
7. Calculate difficulty score.
8. Shuffle question order.
9. Shuffle MCQ option order.
10. Generate Set A, Set B, Set C, etc.
11. Create paper hash.
12. Lock generated paper.
13. Log audit event.

Core Functions

```
generate_paper()  
select_questions_by_blueprint()
```

```
shuffle_questions()
shuffle_options()
validate_paper()
calculate_difficulty_score()
lock_paper()
generate_paper_hash()
```

Generated Paper Object

```
{
  "id": "PPR-001",
  "exam_id": "EXM-001",
  "set_id": "A",
  "blueprint_id": "BLP-001",
  "question_ids": ["QST-101", "QST-245", "QST-389"],
  "option_order_map": {
    "QST-101": ["B", "D", "A", "C"]
  },
  "difficulty_score": 0.71,
  "paper_hash": "sha256_hash",
  "status": "LOCKED"
}
```

4.5 Encryption and Paper Release Module

Responsibility

Encrypts generated papers and releases them only under valid conditions.

Release Conditions

Paper can be released only if:

```
current_time >= exam_start_time
current_time <= exam_release_window_end
paper_status == ENCRYPTED
center_id is authorized
controller approval exists
package_hash is valid
audit_chain is valid
```

Core Functions

```
encrypt_paper()
decrypt_paper()
```

```
generate_exam_package()
validate_release_window()
validate_center_access()
release_paper()
revoke_release()
```

Paper Package Object

```
{
  "package_id": "PKG-001",
  "exam_id": "EXM-001",
  "center_id": "CTR-22",
  "encrypted_paper": "...",
  "package_hash": "sha256_hash",
  "valid_from": "timestamp",
  "valid_until": "timestamp",
  "status": "SEALED"
}
```

4.6 Candidate Verification Module

Responsibility

Verifies candidate identity and binds candidate to an exam session.

Verification Inputs

```
admit_card_qr
candidate_id
government_id_optional
live_photo_optional
signature_optional
center_id
seat_id
```

Verification Status

```
PENDING
VERIFIED
FAILED
MANUAL_REVIEW
BLOCKED
```

Candidate Verification Object

```
{
  "verification_id": "VER-001",
  "candidate_id": "CAN-102",
  "exam_id": "EXM-001",
  "center_id": "CTR-22",
  "seat_id": "A-14",
  "verification_method": ["QR", "PHOTO"],
  "status": "VERIFIED",
  "verified_by": "INV-11",
  "verification_hash": "sha256_hash",
  "timestamp": "timestamp"
}
```

Core Functions

```
scan_admit_card()
verify_candidate_record()
capture_live_photo()
compare_candidate_photo()
assign_seat()
create_verification_hash()
```

4.7 Exam Delivery Module

Responsibility

Provides the candidate-facing exam attempt interface.

CBT Features

```
start_exam()
load_question()
save_answer()
mark_for_review()
clear_response()
auto_save_answer()
submit_exam()
generate_submission_receipt()
```

Candidate Answer Event Object

```
{
  "event_id": "EVT-001",
  "session_id": "SES-001",
  "candidate_id": "CAN-102",
  "question_id": "QST-101",
}
```

```
"selected_answer": "B",
"event_type": "ANSWER_SAVED",
"previous_event_hash": "previous_hash",
"current_event_hash": "current_hash",
"timestamp": "timestamp"
}
```

4.8 OMR Processing Module

Responsibility

Processes scanned OMR sheets and extracts candidate responses.

OMR Pipeline

1. Upload scanned image.
2. Detect QR code.
3. Identify candidate and sheet ID.
4. Preprocess image.
5. Align sheet using markers.
6. Detect bubble regions.
7. Calculate fill percentage.
8. Determine selected answers.
9. Generate confidence score.
10. Flag ambiguous bubbles.
11. Store scan hash.
12. Lock final OMR response.

OMR Result Object

```
{
  "omr_scan_id": "OMRSCAN-001",
  "candidate_id": "CAN-102",
  "exam_id": "EXM-001",
  "sheet_id": "OMR-901",
  "image_hash": "sha256_hash",
  "answers": {
    "Q1": {
      "selected": "A",
      "confidence": 0.98
    },
    "Q2": {
      "selected": "AMBIGUOUS",
      "confidence": 0.52
    }
  }
},
```

```
"status": "MANUAL_REVIEW_REQUIRED"
}
```

Core Functions

```
upload_omr_scan()
detect_qr()
align_omr_image()
detect_bubbles()
calculate_fill_score()
extract_answers()
flag_ambiguous_answers()
lock_omr_response()
```

4.9 Written Answer Evaluation Module

Responsibility

Handles scanned written answer booklets and anonymized evaluation.

Written Booklet Object

```
{
  "booklet_id": "WBK-001",
  "candidate_id": "CAN-102",
  "anonymous_id": "ANON-8842",
  "exam_id": "EXM-001",
  "total_pages": 12,
  "booklet_hash": "sha256_hash",
  "status": "SCANNED"
}
```

Written Page Object

```
{
  "page_id": "WPG-001",
  "booklet_id": "WBK-001",
  "page_number": 1,
  "image_url": "storage_path",
  "page_hash": "sha256_hash",
  "upload_timestamp": "timestamp"
}
```

Evaluation Object

```
{
  "evaluation_id": "EVAL-001",
  "anonymous_id": "ANON-8842",
  "evaluator_id": "EVR-12",
  "question_id": "QST-W1",
  "marks_awarded": 8,
  "max_marks": 10,
  "rubric_notes": "Covered most required points.",
  "status": "LOCKED",
  "evaluation_hash": "sha256_hash"
}
```

Core Functions

```
create_written_booklet()
upload_written_page()
generate_anonymous_id()
assign_evaluator()
submit_marks()
lock_evaluation()
calculate_evaluation_hash()
```

4.10 Result Generation Module

Responsibility

Generates final results after verifying integrity of all required data.

Pre-Result Verification

Before result generation, the system checks:

```
paper_hash_valid == true
candidate_verified == true
answer_submission_hash_valid == true
omr_scan_hash_valid == true
written_page_hash_valid == true
evaluation_locked == true
audit_chain_valid == true
answer_key_locked == true
```

Result Object

```
{
  "result_id": "RES-001",
```

```
"candidate_id": "CAN-102",
"exam_id": "EXM-001",
"marks_obtained": 342,
"max_marks": 400,
"rank": 129,
"status": "QUALIFIED",
"result_hash": "sha256_hash",
"published_at": "timestamp"
}
```

Core Functions

```
verify_exam_integrity()
calculate_mcq_score()
calculate_omr_score()
calculate_written_score()
generate_result()
publish_result()
verify_result()
```

4.11 Audit Ledger Module

Responsibility

Records every sensitive action in a tamper-evident hash chain.

Audit Event Object

```
{
  "event_id": "AUD-001",
  "actor_id": "USR-21",
  "action": "PAPER_GENERATED",
  "resource_type": "GeneratedPaper",
  "resource_id": "PPR-001",
  "payload_hash": "sha256_hash",
  "previous_hash": "previous_audit_hash",
  "current_hash": "current_audit_hash",
  "signature": "digital_signature",
  "timestamp": "timestamp"
}
```

Core Functions

```
create_audit_event()
calculate_payload_hash()
get_previous_audit_hash()
```

calculate_current_hash()
sign_event()
verify_event_signature()
verify_audit_chain()

5. Database Design

5.1 Main Tables

users
roles
user_roles
institutions
exams
subjects
questions
question_versions
paper_blueprints
generated_papers
encrypted_packages
candidates
candidate_verifications
exam_sessions
candidate_answer_events
omr_sheets
omr_scans
written_booklets
written_pages
evaluations
results
audit_logs

5.2 Important Table Schemas

users

id UUID PRIMARY KEY
name VARCHAR
email VARCHAR UNIQUE
password_hash VARCHAR
institution_id UUID
status VARCHAR
created_at TIMESTAMP

updated_at TIMESTAMP

roles

id UUID PRIMARY KEY
name VARCHAR
description TEXT

exams

id UUID PRIMARY KEY
title VARCHAR
exam_type VARCHAR
institution_id UUID
start_time TIMESTAMP
end_time TIMESTAMP
status VARCHAR
created_by UUID
created_at TIMESTAMP
updated_at TIMESTAMP

questions

id UUID PRIMARY KEY
subject_id UUID
topic VARCHAR
difficulty VARCHAR
question_type VARCHAR
marks INT
encrypted_content TEXT
encrypted_answer TEXT
status VARCHAR
author_id UUID
reviewer_id UUID
version INT
content_hash VARCHAR
created_at TIMESTAMP
updated_at TIMESTAMP

paper_blueprints

id UUID PRIMARY KEY
exam_id UUID
total_marks INT
total_questions INT
duration_minutes INT
subject_distribution JSONB

difficulty_distribution JSONB
question_type_distribution JSONB
negative_marking BOOLEAN
negative_marks FLOAT
status VARCHAR
created_at TIMESTAMP
updated_at TIMESTAMP

generated_papers

id UUID PRIMARY KEY
exam_id UUID
blueprint_id UUID
set_id VARCHAR
question_order JSONB
option_order_map JSONB
difficulty_score FLOAT
paper_hash VARCHAR
status VARCHAR
created_at TIMESTAMP

encrypted_packages

id UUID PRIMARY KEY
exam_id UUID
paper_id UUID
center_id UUID
encrypted_payload TEXT
package_hash VARCHAR
valid_from TIMESTAMP
valid_until TIMESTAMP
status VARCHAR
created_at TIMESTAMP

candidates

id UUID PRIMARY KEY
exam_id UUID
name VARCHAR
registration_number VARCHAR
anonymous_id VARCHAR
photo_url TEXT
status VARCHAR
created_at TIMESTAMP

candidate_answer_events

id UUID PRIMARY KEY
session_id UUID
candidate_id UUID
exam_id UUID
question_id UUID
event_type VARCHAR
selected_answer TEXT
previous_event_hash VARCHAR
current_event_hash VARCHAR
created_at TIMESTAMP

omr_scans

id UUID PRIMARY KEY
candidate_id UUID
exam_id UUID
sheet_id UUID
image_url TEXT
image_hash VARCHAR
detected_answers JSONB
confidence_report JSONB
status VARCHAR
created_at TIMESTAMP

written_pages

id UUID PRIMARY KEY
booklet_id UUID
page_number INT
image_url TEXT
page_hash VARCHAR
created_at TIMESTAMP

evaluations

id UUID PRIMARY KEY
exam_id UUID
anonymous_id VARCHAR
evaluator_id UUID
question_id UUID
marks_awarded FLOAT
max_marks FLOAT
rubric_notes TEXT
evaluation_hash VARCHAR
status VARCHAR
created_at TIMESTAMP
updated_at TIMESTAMP

results

id UUID PRIMARY KEY
exam_id UUID
candidate_id UUID
marks_obtained FLOAT
max_marks FLOAT
rank INT
status VARCHAR
result_hash VARCHAR
published_at TIMESTAMP

audit_logs

id UUID PRIMARY KEY
actor_id UUID
action VARCHAR
resource_type VARCHAR
resource_id UUID
payload_hash VARCHAR
previous_hash VARCHAR
current_hash VARCHAR
signature TEXT
created_at TIMESTAMP

6. API Design

6.1 Authentication APIs

POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me
POST /api/auth/assign-role

6.2 Exam APIs

POST /api/exams
GET /api/exams
GET /api/exams/{exam_id}
PATCH /api/exams/{exam_id}
POST /api/exams/{exam_id}/lock
POST /api/exams/{exam_id}/publish-results

6.3 Question Bank APIs

POST /api/questions
GET /api/questions
GET /api/questions/{question_id}
PATCH /api/questions/{question_id}
POST /api/questions/{question_id}/submit
POST /api/questions/{question_id}/review
POST /api/questions/{question_id}/approve
POST /api/questions/{question_id}/reject

6.4 Paper Generation APIs

POST /api/exams/{exam_id}/blueprint
GET /api/exams/{exam_id}/blueprint
POST /api/exams/{exam_id}/generate-paper
POST /api/papers/{paper_id}/encrypt
POST /api/papers/{paper_id}/release
GET /api/papers/{paper_id}/verify

6.5 Candidate APIs

POST /api/candidates/register
GET /api/candidates/{candidate_id}
POST /api/candidates/{candidate_id}/verify
POST /api/candidates/{candidate_id}/assign-seat
GET /api/candidates/{candidate_id}/result

6.6 Exam Session APIs

POST /api/sessions/start
GET /api/sessions/{session_id}/question/{question_id}
POST /api/sessions/{session_id}/answer
POST /api/sessions/{session_id}/mark-review
POST /api/sessions/{session_id}/submit
GET /api/sessions/{session_id}/receipt

6.7 OMR APIs

POST /api/omr/sheets/generate
POST /api/omr/scans/upload
POST /api/omr/scans/{scan_id}/process
POST /api/omr/scans/{scan_id}/manual-review
POST /api/omr/scans/{scan_id}/lock
GET /api/omr/scans/{scan_id}/verify

6.8 Written Evaluation APIs

POST /api/written/booklets/create
POST /api/written/booklets/{booklet_id}/pages
POST /api/written/booklets/{booklet_id}/anonymize
POST /api/evaluations/assign
POST /api/evaluations/{evaluation_id}/submit
POST /api/evaluations/{evaluation_id}/lock
GET /api/evaluations/{evaluation_id}/verify

6.9 Audit APIs

GET /api/audit/logs
GET /api/audit/resource/{resource_type}/{resource_id}
GET /api/audit/verify-chain
GET /api/audit/results/{result_id}/verify

7. UI/UX Design

7.1 UI Design Philosophy

ExamForge should feel like a serious institutional security product, not a casual test app.

The UI should communicate:

- Trust
- Verification
- Auditability
- Clarity
- Control
- Professionalism

Recommended visual direction:

Theme: Institutional security dashboard

Colors: Navy, white, slate, emerald verification states, amber warnings, red critical alerts

Typography: Clean sans-serif

Layout: Dashboard-first

Interaction style: Step-by-step workflows

7.2 Main UI Areas

ExamForge has different dashboards for different roles.

7.2.1 Admin Dashboard

Purpose:

- View all exams
- Monitor system status
- Track active exams
- Review audit health
- Manage users and roles

Main widgets:

Total Exams

Active Exams

Pending Reviews

Paper Release Status

Candidates Verified

Submissions Received

Audit Chain Health

Security Alerts

7.2.2 Exam Controller Dashboard

Pages:

Exam List

Create Exam

Paper Blueprint Builder

Question Pool View

Paper Generation Panel

Encryption Status

Release Control Panel

Result Verification Panel
Publish Result Page

Important UI components:

- Blueprint distribution charts
 - Paper generation status cards
 - Hash preview
 - Lock/unlock state badges
 - Release countdown timer
 - Integrity checklist
-

7.2.3 Question Author Dashboard

Pages:

My Questions
Create Question
Draft Questions
Submitted Questions
Rejected Questions
Version History

Question creation form:

Subject
Topic
Difficulty
Question Type
Marks
Expected Time
Question Content
Options
Correct Answer
Explanation
Tags
Submit for Review Button

7.2.4 Reviewer Dashboard

Pages:

Review Queue
Question Detail

Question Version Diff
Approve / Reject Panel
Reviewer Notes

Reviewer actions:

Approve
Reject
Request Changes
Flag Duplicate
Flag Low Quality

7.2.5 Center Officer Dashboard

Pages:

Center Overview
Candidate Verification
Seat Mapping
Paper Package Status
OMR Upload
Written Booklet Upload
Incident Reports

Important UI states:

Paper Sealed
Release Not Allowed Yet
Release Window Open
Paper Released
Upload Pending
Upload Verified

7.2.6 Candidate Exam UI

Candidate UI must be simple and distraction-free.

Screen layout:

Top Bar:
- Exam name
- Timer
- Candidate anonymous ID
- Submit button

Left Panel:

- Question numbers
- Answered / Not Answered / Marked for Review indicators

Main Panel:

- Question text
- Options
- Save answer
- Clear response
- Mark for review
- Next / Previous

Bottom:

- Submission receipt after final submit

Candidate answer states:

Not Visited

Not Answered

Answered

Marked for Review

Answered and Marked for Review

7.2.7 OMR Processing UI

Pages:

Upload OMR Scan

Scan Preview

Detected Bubble View

Confidence Report

Manual Review Queue

Lock OMR Response

OMR UI components:

Image preview

Detected QR details

Question-wise answer table

Confidence score bars

Ambiguous answer warnings

Manual override panel

Lock response button

7.2.8 Written Evaluation UI

Pages:

Assigned Copies
Answer Booklet Viewer
Rubric Panel
Marks Entry
Evaluation Notes
Lock Evaluation

Evaluator UI layout:

Left:

- Scanned answer page viewer

Right:

- Question selector
- Rubric checklist
- Marks input
- Notes
- Save / Lock buttons

Top:

- Anonymous ID only
 - Candidate identity hidden
-

7.2.9 Auditor Dashboard

Pages:

Audit Ledger
Verify Chain
Resource Audit Trail
Result Verification
Dispute Investigation

Audit UI components:

Hash chain status
Event timeline
Actor/action table
Signature verification badge
Broken chain alert
Result integrity report

8. Page-Level UI Design

8.1 Login Page

Fields:

Email
Password
Role-aware redirect
Login button
Forgot password

Post-login redirects:

Admin → Admin Dashboard
Candidate → Candidate Portal
Evaluator → Evaluation Dashboard
Auditor → Audit Dashboard

8.2 Create Exam Page

Fields:

Exam Title
Exam Type
Exam Mode
Start Time
End Time
Duration
Institution
Subjects
Candidate Count
Exam Centers

Actions:

Save Draft
Create Exam
Lock Exam Config

8.3 Blueprint Builder Page

UI sections:

Total Marks
Total Questions
Subject Distribution
Difficulty Distribution
Question Type Distribution
Negative Marking Rules
Duration
Validation Panel

Validation examples:

Total questions match blueprint: Valid
Marks distribution: Valid
Question pool availability: Warning
Difficulty balance: Valid

8.4 Paper Generation Page

UI sections:

Blueprint Summary
Available Question Pool
Number of Sets
Generate Paper Button
Generated Paper Cards
Difficulty Score
Paper Hash
Lock Paper Button
Encrypt Paper Button

8.5 Release Control Page

UI sections:

Exam Start Time
Release Window
Center Package List
Package Hash
Release Status

Countdown Timer
Approval Checklist
Release Paper Button

States:

Too Early: Release Disabled
Release Window Open: Release Enabled
Expired: Emergency Protocol Required

8.6 Result Verification Page

Integrity checklist:

Paper Hash Valid
Candidate Verification Valid
Submission Hash Valid
OMR Scan Hash Valid
Written Pages Valid
Evaluation Locked
Audit Chain Valid
Answer Key Locked

Actions:

Verify Integrity
Generate Result
Publish Result
Export Result Report

9. Security Design

9.1 Data Security

Sensitive data is encrypted at rest.

Encrypted data:

Question content
Correct answers
Generated papers

Answer keys
Candidate identity documents
Exam packages

9.2 Integrity Security

Every important record has a hash.

Hashed records:

Questions
Papers
Exam packages
Answer events
OMR scans
Written pages
Evaluations
Results
Audit logs

9.3 Audit Security

Every sensitive operation creates an audit event.

Examples:

QUESTION_CREATED
QUESTION_APPROVED
PAPER_GENERATED
PAPER_ENCRYPTED
PAPER_RELEASED
CANDIDATE_VERIFIED
ANSWER_SUBMITTED
OMR_SCANNED
EVALUATION_LOCKED
RESULT_PUBLISHED

9.4 Role-Based Security

Users can only perform actions permitted by their role.

Examples:

Question Author cannot approve own question.
Evaluator cannot see candidate identity.
Center Officer cannot modify paper content.
Candidate cannot access answer key.
Auditor can verify logs but cannot edit exam data.

10. MVP Design

10.1 MVP Goal

The MVP should demonstrate the full trust pipeline in a simplified but convincing way.

The MVP should prove:

A paper can be generated securely.
A paper can be encrypted and released.
A candidate can submit answers.
OMR/written answers can be captured.
Results can be generated only after integrity verification.
Every important action is auditable.

10.2 MVP Modules

Build these modules first:

1. Authentication and roles
 2. Admin dashboard
 3. Question bank
 4. Blueprint builder
 5. Paper generator
 6. Paper encryption simulation
 7. Candidate MCQ exam portal
 8. Answer submission hash
 9. OMR scan upload and mock processing
 10. Written answer upload and anonymous evaluation
 11. Result verification
 12. Audit ledger
-

10.3 MVP Screens

Required screens:

- Login
- Admin Dashboard
- Create Exam
- Question Bank
- Create Question
- Blueprint Builder
- Generate Paper
- Encrypt and Release Paper
- Candidate Exam Portal
- Submission Receipt
- OMR Upload
- Written Evaluation
- Result Verification
- Audit Ledger

10.4 MVP Technical Stack

Frontend:

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui

Backend:

- FastAPI
- Python
- SQLAlchemy
- SQLite for MVP
- PostgreSQL for production

Security:

- SHA-256 hashing
- AES-GCM encryption
- JWT authentication
- Role-based access control
- Hash-chained audit logs

Computer Vision:

- OpenCV for OMR prototype

Deployment:

Docker

Vercel for frontend

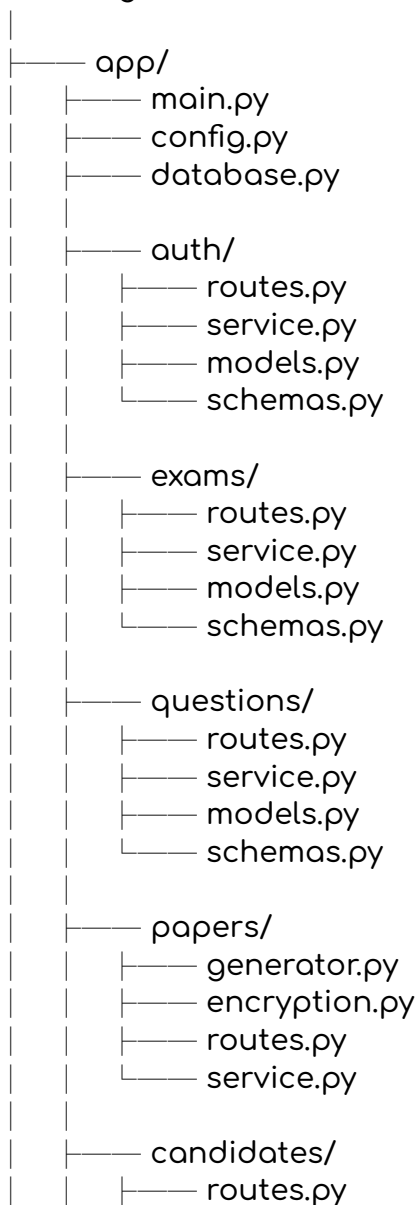
Render/Railway/Fly.io for backend

SQLite/PostgreSQL database

11. Suggested Folder Structure

11.1 Backend Folder Structure

examforge-backend/

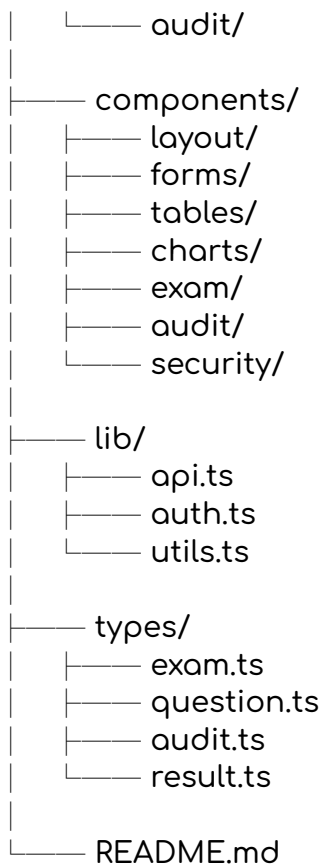


```
|
|
|   |— service.py
|   |— models.py
|   |— schemas.py
|
|   |— omr/
|       |— processor.py
|       |— routes.py
|       |— service.py
|
|   |— written/
|       |— routes.py
|       |— service.py
|       |— models.py
|
|   |— results/
|       |— routes.py
|       |— service.py
|
|   |— audit/
|       |— ledger.py
|       |— routes.py
|       |— service.py
|
|   |— security/
|       |— hashing.py
|       |— encryption.py
|       |— signatures.py
|
|— tests/
|— requirements.txt
|— README.md
```

11.2 Frontend Folder Structure

examforge-frontend/

```
|
|— app/
|   |— login/
|   |— dashboard/
|   |— exams/
|   |— questions/
|   |— blueprint/
|   |— papers/
|   |— candidate/
|   |— omr/
|   |— written-evaluation/
|   |— results/
```



12. Non-Functional Requirements

12.1 Security

All sensitive data must be encrypted.
All critical actions must be logged.
All result generation must be verifiable.
Users must only access role-permitted actions.

12.2 Reliability

Candidate answers should auto-save.
Exam sessions should survive page refresh.
OMR scans should not overwrite locked responses.
Results should not generate if integrity checks fail.

12.3 Scalability

System should support multiple institutions.

System should support multiple exams.

Paper generation should work for large question banks.

Audit logs should be append-only.

12.4 Usability

Candidate UI should be simple.

Admin workflows should be step-by-step.

Security state should be visible through clear badges.

Errors should explain what failed and why.

13. Future Design Extensions

Future versions can include:

Threshold cryptography

Hardware security module integration

Offline center desktop app

Face verification

Secure printing support

Advanced AI answer evaluation assistant

Anomaly detection for cheating patterns

Blockchain anchoring for public audit snapshots

Dispute resolution portal

Multi-language question paper support

Large-scale national exam architecture

14. Conclusion

ExamForge is designed as a modular, secure, and format-agnostic examination infrastructure.

The High-Level Design defines the major services and data flow. The Low-Level Design defines the internal modules, objects, functions, and workflows. The UI Design defines role-based dashboards and step-by-step user interactions.

Together, the design supports the central mission of ExamForge:

To make examination integrity technically verifiable across paper creation, exam delivery, answer capture, evaluation, result generation, and audit.